
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON LUNG - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-IV-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-IV.bin          efc9991c3114d51994aa0dc080bbcb592bc9432c7739b850eeb63e73e7e62316
ACT-IV.uf2          f7e79d0b0e3104e3ee0f3a2c032d9ba95da2d1128235ffa8b822d74c6b8542be
ACT-IV_fixed.bin    7d59dc65e512c30630f51b20baf1c6e358ae54a024e88cfd8373d847ea170f7c
ACT-IV_fixed.uf2    977c4cad1f60e05060fd933050ecc7c0f04e319cbee9c3c441c41c9e37137775
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-IV.bin` and `ACT-IV_fixed.bin` SHA-256 values. Both `.bin` images are 51,012 bytes and both `.uf2` images are 102,912 bytes.

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Persistence re-install	<code>implant_init</code> (inlined <code>implant_resume</code>)	<code>0xA46F</code>	<code>0x1000A46F</code>	<code>0xD1</code>	<code>0xD0</code>
2 Rootkit hiding	<code>implant_rootkit_active</code>	<code>0xA2D1</code>	<code>0x1000A2D1</code>	<code>0xB9</code>	<code>0xB1</code>
3 Reserved-sector write	<code>implant_init</code> (inlined <code>implant_persist</code>)	<code>0xA42D</code>	<code>0x1000A42D</code>	<code>0xB9</code>	<code>0xB1</code>
4 SETPOINT authorization	<code>control_handle_frame</code>	<code>0x7581</code>	<code>0x10007581</code>	<code>0xD1</code>	<code>0xD0</code>

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-IV.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronLung_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` (`VA = offset + 0x10000000`).

Vector Table Decoding. First 32 bytes of `ACT-IV.bin`:

```
00 20 08 20 5D 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

Evidence	Answer
Vector table base	<code>0x10000000</code>
Initial SP	<code>0x20082000</code>

Evidence	Answer
Reset handler (as stored)	0x1000015D
Reset instruction address	0x1000015C

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry 0x1000015C.

Entry and Monitor Loop. From ACT-IV-main-disasm.txt :

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fa93     bl      10003760 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada     bl      100037f4 <__wrap_puts>
10000240:    f006 f8fa     bl      10006438 <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 f9cb     bl      100065e0 <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>

```

Element	Address
main	0x10000234
monitor_init	0x10006438
monitor_step	0x100065E0

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / HVAC state machine	monitor_init	0x10006438
Monitor / HVAC state machine	monitor_step	0x100065E0
Control (sealed SETPOINT path)	control_handle_frame	0x1000750C
SETPOINT authorization	hvac_auth_apply	0x100076B8
Baffle (actuator)	baffle_close	0x1000A83C
Implant	implant_rootkit_active	0x1000A2C8
Implant	implant_tick	0x1000A2FC
Implant	implant_handle_command	0x1000A3CC
Implant	implant_init	0x1000A404
Crypto	envelope_open_hex	0x10007960
Crypto	crypto_aead_seal	0x100077D0
Crypto	crypto_aead_tag_equal	0x10007784

Module	Anchor function	Address
Radio	radio_send_frame	0x1000A534

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project IronLung_Investigation , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base 0x10000000
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base 0x10000000 , initial SP 0x20082000 , reset handler 0x1000015D
[DOCUMENT] main and the HVAC controller state machine (monitor_step) addresses identified	1	main 0x10000234 , monitor_step 0x100065E0
[DOCUMENT] Module map identifies the baffle, control, hvac_auth, implant, and monitor anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used Raw Binary , ARM Cortex 32 little endian default , base 0x10000000 , and that auto-analysis completed before any address was read. In the language dialog the student must search Cortex and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler 0x1000015D is odd because bit 0 selects Thumb; clearing it gives 0x1000015C .
- Always say reset handler , never reset pointer .
- The vector table is identical in the compromised and corrected images because no defect touches it.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient.

Task 2: Bug #1 The Persistence Re-Install (20 points)

Solution

Locate the branch. In `implant_init` (starts at 0x1000A404) the re-install gate is at file offset 0xA46F (VA 0x1000A46F). The corrected image is:

```

1000a404 <implant_init>:
1000a404: 2300      movs    r3, #0
1000a406: b5f0      push    {r4, r5, r6, r7, lr}
1000a408: 4a1c      ldr     r2, [pc, #112] @ (1000a47c <implant_init+0x78>)
1000a40a: 4e1d      ldr     r6, [pc, #116] @ (1000a480 <implant_init+0x7c>)
1000a40c: 4d1d      ldr     r5, [pc, #116] @ (1000a484 <implant_init+0x80>)
1000a40e: 4c1e      ldr     r4, [pc, #120] @ (1000a488 <implant_init+0x84>)
1000a410: 4f1e      ldr     r7, [pc, #120] @ (1000a48c <implant_init+0x88>)
1000a412: 491f      ldr     r1, [pc, #124] @ (1000a490 <implant_init+0x8c>)
1000a414: 481f      ldr     r0, [pc, #124] @ (1000a494 <implant_init+0x90>)
1000a416: b0c1      sub     sp, #260 @ 0x104
1000a418: 7013      strb    r3, [r2, #0]
1000a41a: 6033      str     r3, [r6, #0]
1000a41c: 703b      strb    r3, [r7, #0]
1000a41e: 702b      strb    r3, [r5, #0]
1000a420: 6023      str     r3, [r4, #0]
1000a422: 700b      strb    r3, [r1, #0]
1000a424: 7803      ldrb    r3, [r0, #0]
1000a426: 2bc7      cmp     r3, #199 @ 0xc7
1000a428: d01f      beq.n   1000a46a <implant_init+0x66>
1000a42a: 780b      ldrb    r3, [r1, #0]
1000a42c: b1db      cbz     r3, 1000a466 <implant_init+0x62>
1000a42e: 7803      ldrb    r3, [r0, #0]
1000a430: 2bc7      cmp     r3, #199 @ 0xc7
1000a432: d018      beq.n   1000a466 <implant_init+0x62>
1000a434: f3ef 8410 mrs     r4, PRIMASK
1000a438: b672      cpsid   i
1000a43a: 22ff      movs    r2, #255 @ 0xff
1000a43c: f10d 0001 add.w    r0, sp, #1
1000a440: 4611      mov     r1, r2
1000a442: f000 fa47 bl      1000a8d4 <memset>
1000a446: 23c7      movs    r3, #199 @ 0xc7
1000a448: f44f 5180 mov.w    r1, #4096 @ 0x1000
1000a44c: 4812      ldr     r0, [pc, #72] @ (1000a498 <implant_init+0x94>)
1000a44e: f88d 3000 strb.w   r3, [sp]
1000a452: f000 fb8d bl      1000ab70 <__flash_range_erase_veneer>
1000a456: f44f 7280 mov.w    r2, #256 @ 0x100
1000a45a: 4669      mov     r1, sp
1000a45c: 480e      ldr     r0, [pc, #56] @ (1000a498 <implant_init+0x94>)
1000a45e: f000 fb6b bl      1000ab38 <__flash_range_program_veneer>
1000a462: f384 8810 msr     PRIMASK, r4
1000a466: b041      add     sp, #260 @ 0x104
1000a468: bdf0      pop     {r4, r5, r6, r7, pc}
1000a46a: 7813      ldrb    r3, [r2, #0]
1000a46c: 2b00      cmp     r3, #0
1000a46e: d0fa      beq.n   1000a466 <implant_init+0x62>
1000a470: 2201      movs    r2, #1
1000a472: 2303      movs    r3, #3
1000a474: 702a      strb    r2, [r5, #0]
1000a476: 6023      str     r3, [r4, #0]
1000a478: b041      add     sp, #260 @ 0x104
1000a47a: bdf0      pop     {r4, r5, r6, r7, pc}
1000a47c: 20013cf9 .word    0x20013cf9
1000a480: 20013710 .word    0x20013710
1000a484: 20013cf7 .word    0x20013cf7
1000a488: 20013714 .word    0x20013714
1000a48c: 20013cfa .word    0x20013cfa
1000a490: 20013cf8 .word    0x20013cf8
1000a494: 103ff000 .word    0x103ff000
1000a498: 003ff000 .word    0x003ff000

```

Instruction decode. `ldr r2, [pc, #112]` loads the re-install gate at `0x20013CF9`, and the init prologue clears it at `0x1000A422`. The reserved sector comes from the literal at `0x1000A494` (`0x103FF000`), and the marker is read as a byte by `ldrb r3, [r0, #0]` at `0x1000A424`. `cmp r3, #199` tests the marker against `0xC7`. When the marker is present the code branches to `0x1000A46A`, reads the re-install gate with `ldrb r3, [r2, #0]`, compares it to zero, and the branch at `0x1000A46E` decides whether the implant re-arms. The correct code leaves the implant

disarmed when the gate is clear, so the branch at `0x1000A46E` must be `beq (0xD0)` to the `0x1000A466` return. When the gate is set, the code falls through to `implant_arm`: it sets the arming flag at `0x20013CF7` and the trigger tick at `0x20013714` to 3 (the tick counter is zero at boot, so `0 + IMPLANT_TRIGGER_DELAY_TICKS` is a literal 3). The condition byte is the high byte at `0x1000A46F`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A46F</code>	<code>0xA46F</code>	<code>0xD1</code>	<code>bne.n 0x1000A466</code>	<code>0xD0</code>	<code>beq.n 0x1000A466</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA46F</code>	<code>0x1000A46F</code>	FA D1	FA D0

Why the implant no longer re-installs. Under the compromised `bne`, the re-install gate is inverted: the re-arm path is taken when the gate is clear, so on every boot with the marker present the implant re-arms its beacon and bomb. After the patch, `beq` returns while the gate is clear, so the marker never re-arms the implant. Because the reserved sector at `0x103FF000` sits outside the program region that a firmware reflash writes, the marker survives the reflash; without this patch the loader keeps finding it and the payload keeps coming back. Erasing the reserved sector removes the copy, and this patch stops the loader from acting on a future copy. There is no single patch for persistence: remove the loader and remove the payload.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the persistence re-install branch at <code>0x1000A46F</code>	5	Address and function (<code>implant_init</code>) identified
[DOCUMENT] Documented the reserved-sector marker and the re-install branch	5	<code>0x103FF000</code> , marker <code>0xC7</code> , a present marker re-arms on boot
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so a present marker no longer re-arms the implant	7	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Explained that a firmware reflash alone does not remove the implant	3	The reserved sector is outside the program region

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA46F`; the correct halfword is `d0fa` for `beq.n` and the compromised halfword is `d1fa`, so the on-disk bytes are `FA D0` for the fix and `FA D1` for the compromise.
- `beq` branches when the comparison result is equal (the gate is zero); `bne` branches when it is not equal. The register holds the re-install gate, so the semantics are "skip the re-arm when the gate is clear".
- The re-install path is inlined. There is no standalone `implant_resume` symbol in the stripped image; the path lives at the tail of `implant_init`.
- The reserved sector is `HVAC_IMPLANT_RESERVE_ADDR (0x103FF000)` and the marker byte is `IMPLANT_MARKER_BYTE (0xC7)`.
- Full credit requires both the byte change and a correct statement of the reserved-sector persistence lesson.
- Remind students that the fix is two parts: the patch stops the re-install, and erasing the reserved sector removes the marker a future image could read.

Task 3: Bug #2 The Rootkit Hiding (20 points)

Solution

Locate the branch. `implant_rootkit_active` starts at `0x1000A2C8` and the rootkit gate is at file offset `0xA2D1` (VA `0x1000A2D1`). The corrected image is:

```

1000a2c8 <implant_rootkit_active>:
1000a2c8: 4b0a      ldr     r3, [pc, #40]    @ (1000a2f4 <implant_rootkit_active+0x2c>)
1000a2ca: 781b      ldrb    r3, [r3, #0]
1000a2cc: f003 00ff  and.w   r0, r3, #255    @ 0xff
1000a2d0: b123      cbz     r3, 1000a2dc <implant_rootkit_active+0x14>
1000a2d2: 4b09      ldr     r3, [pc, #36]    @ (1000a2f8 <implant_rootkit_active+0x30>)
1000a2d4: 781b      ldrb    r3, [r3, #0]
1000a2d6: 2bc7      cmp     r3, #199        @ 0xc7
1000a2d8: d001      beq.n   1000a2de <implant_rootkit_active+0x16>
1000a2da: 2000      movs    r0, #0
1000a2dc: 4770      bx      lr
1000a2de: f04f 23e0  mov.w   r3, #3758153728 @ 0xe000e000
1000a2e2: f8d3 0df0  ldr.w   r0, [r3, #3568] @ 0xdf0
1000a2e6: f000 0003  and.w   r0, r0, #3
1000a2ea: fab0 f080  clz     r0, r0
1000a2ee: 0940      lsrs    r0, r0, #5
1000a2f0: 4770      bx      lr
1000a2f2: bf00      nop
1000a2f4: 20013cfa .word   0x20013cfa
1000a2f8: 103ff000 .word   0x103ff000

```

Instruction decode. `ldr r3, [pc, #40]` loads the rootkit gate at `0x20013CFA`, and `ldrb r3, [r3, #0]` reads it. The branch at `0x1000A2D0` decides whether the rootkit can be active. The correct code returns false when the gate is clear, so the branch at `0x1000A2D0` must be `cbz (0xB1)` to the `0x1000A2DC` return. When the gate is set, the code checks the reserved-sector marker at `0x103FF000` (`cmp r3, #199, 0xC7`) and then reads CoreDebug DHCSR at `0xE000EDF0` to confirm no probe is attached. The condition byte is the high byte at `0x1000A2D1`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A2D1</code>	<code>0xA2D1</code>	<code>0xB9</code>	<code>cbnz r3, 0x1000A2DC</code>	<code>0xB1</code>	<code>cbz r3, 0x1000A2DC</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA2D1</code>	<code>0x1000A2D1</code>	23 B9	23 B1

Why the node is no longer hiding. `implant_rootkit_active` returns true when the marker is present and no probe is attached. In `monitor.c`, while the rootkit is active, `monitor_beacon_text` renders the beacon field as `--` instead of `UP`, and `monitor_log` suppresses the `BCN` maintenance line. The beacon still transmits on the radio. Under the compromised `cbnz`, the gate is inverted: the masking path is taken when the gate is clear, so the operator reads a clean node while the radio is still talking. After the patch, `cbz` returns false while the gate is clear, so the LCD shows `B:UP` and the `BCN` line returns even though the beacon is live. A rootkit is not about hiding the code; it is about controlling what the defender is allowed to observe.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the rootkit hiding branch at 0x1000A2D1	5	Address and function (<code>implant_rootkit_active</code>) identified
[DOCUMENT] Documented the LCD and log masking (<code>B:UP</code> to <code>B:--</code> , <code>BCN</code> suppressed) while the beacon transmits	5	Correct masking detail
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the rootkit gate is closed	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained how to prove the beacon status is visible again	3	<code>B:UP</code> and the <code>BCN</code> line return while the rootkit gate is clear

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA2D1` ; the correct halfword is `b123` for `cbz` and the compromised halfword is `b923` , so the on-disk bytes are `23 B1` for the fix and `23 B9` for the compromise.
- `cbz` branches when the register is zero; `cbnz` branches when it is non-zero. The register holds the rootkit gate byte, so the semantics are "return false when the gate is clear".
- The masking is in `src/monitor.c`: `monitor_beacon_text` returns `--` when `monitor_implant_rootkit()` is true, and `monitor_log` prints `BCN` only when the rootkit is not active.
- The anti-debug interacts here: `implant_rootkit_active` returns false while a probe is attached, so the `B:UP` and `BCN` evidence returns the moment a debugger is connected. This is why the rootkit looks dormant on the bench until the anti-debug is defeated.
- Full credit requires both the byte change and a correct statement of the visibility-control lesson.

Task 4: Bug #3 The Reserved-Sector Write (20 points)

Solution

Locate the branch. The `implant_persist` path is inlined into `implant_init` (starts at `0x1000A404`). The persist gate is at file offset `0xA42D` (VA `0x1000A42D`). The corrected image is:

```

1000a424:      7803      ldrb    r3, [r0, #0]
1000a426:      2bc7      cmp     r3, #199      @ 0xc7
1000a428:      d01f      beq.n   r3, #199      @ 0xc7
1000a42a:      780b      ldrb    r3, [r1, #0]
1000a42c:      b1db      cbz     r3, 1000a466 <implant_init+0x62>
1000a42e:      7803      ldrb    r3, [r0, #0]
1000a430:      2bc7      cmp     r3, #199      @ 0xc7
1000a432:      d018      beq.n   r3, #199      @ 0xc7
1000a434:      f3ef 8410    mrs     r4, PRIMASK
1000a438:      b672      cpsid   i
1000a43a:      22ff      movs    r2, #255      @ 0xff
1000a43c:      f10d 0001    add.w   r0, sp, #1
1000a440:      4611      mov     r1, r2
1000a442:      f000 fa47      bl      1000a8d4 <memset>
1000a446:      23c7      movs    r3, #199      @ 0xc7
1000a448:      f44f 5180    mov.w   r1, #4096      @ 0x1000
1000a44c:      4812      ldr     r0, [pc, #72]  @ (1000a498 <implant_init+0x94>)
1000a44e:      f88d 3000    strb.w  r3, [sp]
1000a452:      f000 fb8d    bl      1000ab70 <__flash_range_erase_veneer>
1000a456:      f44f 7280    mov.w   r2, #256      @ 0x100
1000a45a:      4669      mov     r1, sp
1000a45c:      480e      ldr     r0, [pc, #56]  @ (1000a498 <implant_init+0x94>)
1000a45e:      f000 fb6b    bl      1000ab38 <__flash_range_program_veneer>
1000a462:      f384 8810    msr     PRIMASK, r4
1000a466:      b041      add     sp, #260      @ 0x104
1000a468:      bdf0      pop     {r4, r5, r6, r7, pc}

```

Instruction decode. `ldrb r3, [r0, #0]` reads the reserved-sector marker at `0x103FF000` (literal at `0x1000A494`), and `cmp r3, #199` tests the marker against `0xC7`. If the marker is already present, the code returns. Otherwise `ldrb r3, [r1, #0]` reads the persist gate at `0x20013CF8`. The correct code writes no marker when the gate is clear, so the branch at `0x1000A42C` must be `cbz` (`0xB1`) to the `0x1000A466` return. When the gate is set, a second marker check guards the write, and the marker byte `0xC7` is written once through the flash API: `flash_range_erase` at `0x1000A452` erases the sector and `flash_range_program` at `0x1000A45E` programs the marker page. The condition byte is the high byte at `0x1000A42D`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A42D	0xA42D	0xB9	cbnz r3, 0x1000A466	0xB1	cbz r3, 0x1000A466

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA42D	0x1000A42D	DB B9	DB B1

The anti-debug obstacle. The implant reads CoreDebug DHCSR at `0xE000EDF0` in `implant_tick` and returns early while a probe is attached, which suppresses the beacon and the detonation path:

```

1000a30c:      f8dc 2df0    ldr.w   r2, [ip, #3568] @ 0xdf0
1000a310:      0791      lsls    r1, r2, #30
1000a312:      d109      bne.n   r1, #0, 1000a328 <implant_tick+0x2c>

```

The shift keeps bit 1 (`C_HALT`) and bit 0 (`C_DEBUGEN`) and discards the rest; a non-zero result means a probe is attached and the tick returns early. The same register is read again at `0x1000A33A` to stamp the beacon blob. The guard is identical in both images, so it is an analysis obstacle, not one of the four graded defects.

Defeating the anti-debug. Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes `implant_debug_attached` return false for the tick. Show the command sequence, not a fabricated transcript:

```
arm-none-eabi-gdb ACT-IV.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_tick
(gdb) continue
(gdb) set {unsigned int}0xE000EDF0 = 0
(gdb) continue
(gdb) break *0x1000A45E
(gdb) continue
(gdb) x/4xb 0x103FF000
```

To observe the boot write, break on the flash program call at `0x1000A45E` (`bl __flash_range_program_veneer`) in `implant_init`, then read the reserved sector at `0x103FF000`. To observe the beacon and the bomb, clear the debug bits (or patch the `ldr.w` at `0x1000A30C` in a scratch copy to load a zero constant) and let `implant_tick` run. The scratch copy is for observation only; the shipped artifact is patched at the defect.

Why no marker is written. Under the compromised `cbnz`, the persist gate is inverted: the marker write path is taken when the gate is clear, so the first boot writes `0xC7` to `0x103FF000`. After the patch, `cbz` returns while the gate is clear, so the flash program call at `0x1000A45E` is never reached and the reserved sector stays blank. The same gate also covers the detonation-path copy in `implant_tick`, which stays gated in both images. The marker is the state that lets the re-install branch in Task 2 find a payload, which is why both fixes matter.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the reserved-sector persist branch at <code>0x1000A42D</code>	5	Address and inlined <code>implant_init</code> path identified
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so no marker is written to <code>0x103FF000</code>	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the write-once marker byte <code>0xC7</code>	3	Marker, reserved sector, write-once first run

Instructor Notes & Assembly

- The persist path is inlined into `implant_init`; there is no standalone `implant_persist` symbol in the stripped image.
- The condition byte is the high byte at `0xA42D`; the correct halfword is `b1db` for `cbz` and the compromised halfword is `b9db`, so the on-disk bytes are `DB B1` for the fix and `DB B9` for the compromise.
- The marker byte is `IMPLANT_MARKER_BYTE` (`0xC7`), the reserved sector is `HVAC_IMPLANT_RESERVE_ADDR` (`0x103FF000`), and the persist gate is at `0x20013CF8`.
- The DHCSR address is `HVAC_IMPLANT_DHCSR_ADDR` (`0xE000EDF0`); bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT`. The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.

- A common failure is patching the shipped artifact at `0xA42D` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.

Task 5: Bug #4 The SETPOINT Authorization (20 points)

Solution

Locate the branch. In `control_handle_frame` (starts at `0x1000750C`) the authorization branch is at file offset `0x7581` (VA `0x10007581`). The corrected image is:

```

10007574:    990a        ldr    r1, [sp, #40]    @ 0x28
10007576:    4807        ldr    r0, [pc, #28]    @ (10007594 <control_handle_frame+0x88>)
10007578:    aa06        add    r2, sp, #24
1000757a:    f000 f89d    bl     100076b8 <hvac_auth_apply>
1000757e:    2800        cmp    r0, #0
10007580:    d0e1        beq.n 10007546 <control_handle_frame+0x3a>
10007582:    4b05        ldr    r3, [pc, #20]    @ (10007598 <control_handle_frame+0x8c>)
10007584:    801c        strh   r4, [r3, #0]
10007586:    b016        add    sp, #88 @ 0x58
10007588:    bd10        pop    {r4, pc}

```

Instruction decode. After the sealed frame is opened and the command byte is range-checked, `hvac_auth_apply` verifies the anti-replay sequence window and the authenticated-state tag and returns its authorization verdict in `r0`. `cmp r0, #0` tests the verdict, and the branch at `0x10007580` decides whether the command may reach the applied setpoint. The correct code rejects a failed or replayed authorization, so the branch at `0x10007580` must be `beq (0xD0)` to the `0x10007546` reject path, which returns zero. Only a true verdict falls through to `strh r4, [r3, #0]`, which writes the decoded setpoint at `0x20013CF3`. The condition byte is the high byte at `0x10007581`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x10007581</code>	<code>0x7581</code>	<code>0xD1</code>	<code>bne.n 0x10007546</code>	<code>0xD0</code>	<code>beq.n 0x10007546</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x7581</code>	<code>0x10007581</code>	E1 D1	E1 D0

Why the setpoint now requires authorization. Under the compromised `bne`, the verdict is inverted: a failed or replayed authorization falls through to the store at `0x10007584`, while a genuine authorization branches to the reject path and returns zero. After the patch, `beq` sends a false verdict to the reject path at `0x10007546`, so an unauthenticated command, a forged command, and a replayed captured command all fail before the setpoint changes. A legitimate authorized command still returns true and applies. The authorization verdict is the last gate before the setpoint is trusted; inverting it is worse than deleting it, because the machine now acts on exactly the commands it should refuse. The rest of the path is correct: the envelope is opened under the field key, the command byte is checked against `HVAC_COMMAND_SETPoint` (`0x01`), the setpoint is checked against the band `[50, 350]`, and the sequence window and state tag run.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the SETPOINT authorization branch at 0x10007581	5	Address and function (control_handle_frame) identified
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so unauthenticated and replayed commands are rejected	7	Byte 0xD1 changed to 0xD0
[DOCUMENT] Explained that unauthenticated and replayed SETPOINT commands must be rejected	3	The actuator decision must see only an authorized verdict

Instructor Notes & Assembly

- The condition byte is the high byte at 0x7581 ; the correct halfword is d0e1 for beq.n and the compromised halfword is d1e1 , so the on-disk bytes are E1 D0 for the fix and E1 D1 for the compromise.
- hvac_auth_apply performs the monotonic anti-replay check and the authenticated-state tag, so this branch is the verdict for both freshness and state integrity.
- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the setpoint command path is correct. Only the verdict seam was broken.
- This is the defect that is a policy defect rather than an implant behavior, and it is the one a defender would fix first in production.

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, File -> Export Program... , choose Binary Format , and save as ACT-IV_fixed.bin . The shipped image is 51,012 bytes.

Convert.

```
python uf2conv.py ACT-IV_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-IV_fixed.uf2
```

If uf2conv.py is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-IV is 102,912 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

Hardware proof. Flash `ACT-IV_fixed.uf2` in BOOTSEL mode and confirm:

- the reserved sector at `0x103FF000` stays blank after a boot;
- a boot with a marker present no longer re-arms the beacon or the bomb;
- the LCD shows `B:UP` and the log prints `BCN` while the rootkit gate is clear;
- an unauthenticated command and a replayed captured command are rejected before the setpoint changes;
- a legitimate authorized command still applies, and the manual override and the fail-safe still behave.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	The Persistence Re-Install	<code>0xA46F</code>	<code>0x1000A46F</code>	<code>D1</code>	<code>D0</code>
2	The Rootkit Hiding	<code>0xA2D1</code>	<code>0x1000A2D1</code>	<code>B9</code>	<code>B1</code>
3	The Reserved-Sector Write	<code>0xA42D</code>	<code>0x1000A42D</code>	<code>B9</code>	<code>B1</code>
4	The SETPOINT Authorization	<code>0x7581</code>	<code>0x10007581</code>	<code>D1</code>	<code>D0</code>

Reflection mapping. The four defects map to real control-system failures:

Defect	Real-world failure
The Persistence Re-Install	A payload stores state outside the program region, so the standard reflash remediation does not remove it.
The Rootkit Hiding	A compromised controller masks its own traffic from the operator display and the log while the radio keeps transmitting.
The Reserved-Sector Write	A payload writes a durable marker to a reserved sector, so the state that re-arms it survives remediation.
The SETPOINT Authorization	An inverted verdict lets an unauthenticated or replayed command change a physical setpoint.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported <code>ACT-IV_fixed.bin</code> from Ghidra	2	Valid patched binary
[PATCH] Converted to <code>ACT-IV_fixed.uf2</code> with the correct base and family	2	<code>--base 0x10000000 --family 0xe48bff59</code>
[DOCUMENT] <code>scripts/verify_ctf.py</code> passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from `ACT-IV.bin` in exactly the four bytes in the table; `scripts/verify_ctf.py` checks this and the SHA-256 values.

- Confirm the UF2 conversion used base `0x10000000` and family `0xe48bff59`.
- The shipped image is 51,012 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 room climate sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HVAC ALARM, 220 to 330 ohm to GND
Yellow LED	anode	GP17	OVERRIDE PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	HVAC NOMINAL, 220 to 330 ohm to GND
Manual override button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10

Task	Title	Points
Task 2	Bug #1 The Persistence Re-Install	20
Task 3	Bug #2 The Rootkit Hiding	20
Task 4	Bug #3 The Reserved-Sector Write	20
Task 5	Bug #4 The SETPOINT Authorization	20
Task 6	Export and Verify	10
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational building-management system, a clean-room control system, a pharmaceutical network, a public network, a military system, or a third-party device.

Common Student Mistakes

- Patching the low byte of the branch at `0xA46E`, `0xA2D0`, `0xA42C`, or `0x7580` instead of the condition byte at `0xA46F`, `0xA2D1`, `0xA42D`, or `0x7581`.
- Reading the re-install gate backwards and believing the corrected build still re-arms.
- Searching for a standalone `implant_persist` or `implant_resume` symbol and missing that both are inlined into `implant_init`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the marker write, so the payload is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the two clearing gates.
- Forgetting that the fix for persistence is two parts: the patch and the reserved-sector erasure.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.
- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award partial credit for naming the reserved sector and the marker without the persistence lesson, or the lesson without the addresses.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```

--- ACT-IV.bin (compromised)
+++ ACT-IV_fixed.bin (corrected)

Offset 0x00007581: D1 -> D0 (bne.n 0x10007546 -> beq.n 0x10007546)
Offset 0x0000A2D1: B9 -> B1 (cbnz r3, 0x1000A2DC -> cbz r3, 0x1000A2DC)
Offset 0x0000A42D: B9 -> B1 (cbnz r3, 0x1000A466 -> cbz r3, 0x1000A466)
Offset 0x0000A46F: D1 -> D0 (bne.n 0x1000A466 -> beq.n 0x1000A466)

```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	The Persistence Re-Install	0xA46F	0x1000A46F	FA D1	FA D0
2	The Rootkit Hiding	0xA2D1	0x1000A2D1	23 B9	23 B1
3	The Reserved-Sector Write	0xA42D	0x1000A42D	DB B9	DB B1
4	The SETPOINT Authorization	0x7581	0x10007581	E1 D1	E1 D0

Four defects, four changed bytes in four instructions: the re-install gate, the rootkit masking gate, the reserved-sector persist gate, and the authorization verdict. No other byte in either image differs.